DripJar Phase 4A

Final Hardening Report

Fee Rounding Correction · Ledger FK NOT NULL · August 4, 2026

This report documents the Phase 4A hardening corrections applied on top of the financial ledger foundation: an explicit half-up fee rounding formula and enforcement of the ledger_transactions.financial_transaction_id NOT NULL constraint.

Starting HEAD (base for this work)

b6fd453 — Update agent memory with Phase 4A learnings

% % 8841ec9 — Add DripJar financial ledger foundation

% % 65de175 — Add M3Jar cutoff and reminder automation

Final Commit

d63d0f9 — Phase 4A hardening: fee rounding + ledger FK NOT NULL

Pushed to: https://github.com/JB102298/TripJar.git (origin/main)

' Constraints Confirmed

No Stripe / payment-provider integration added

No production DB, DNS, Resend, Cloudflare, secrets, or deployment touched

postCommitPrincipal NOT exposed through any HTTP route

fund_destination_type unchanged — remains NULL in Phase 4A

1 · Files Changed

File	Change
artifacts/api-server/src/lib/fee-engine.ts	Replace Math.round with explicit half-up Math.floor formula; update JSDoc + examples
artifacts/api-server/src/lib/ledger.ts	Make financialTransactionId required in PostLedgerTransactionInput; add guard; remove ?? null fallback
artifacts/api-server/src/__tests__/phase4a-ledger.test.ts	Add midpoint/below/above/large fee tests; fix 3 invariant test calls; add FK NOT NULL invariant
lib/db/src/schema/index.ts	Add .notNull() to ledgerTransactions.financialTransactionId; update comment
lib/db/drizzle/0009_ledger_tx_fk_not_null.sql	NEW — idempotent migration with orphan-safety gate
lib/db/drizzle/meta/_journal.json	Register migration 0009 (idx 9, when 1785862200000)

2 · Fee Rounding Implementation

Change

The original implementation used Math.round(), which relies on JavaScript's internal floating-point representation of the division result. For fee values at exact half-cent boundaries the behavior is correct in practice, but the formula is not transparently deterministic.

The corrected formula uses entirely integer arithmetic:

```
// OLD
return Math.round((principalCents * rateBps) / 10_000);

// NEW — explicit half-up integer arithmetic
return Math.floor((principalCents * rateBps + 5_000) / 10_000);
```

How It Works

Adding 5_000 (half of the divisor 10_000) before the floor division guarantees that values at exactly the half-cent boundary round UP. Values below the boundary round DOWN. There is no floating-point division until after the addition, and Math.floor on an integer-valued double is exact.

Principal (¢)	Exact fee (¢)	Result	Case
50	1.5	2	Midpoint — rounds UP
150	4.5	5	Midpoint — rounds UP

500

15.0

			Exact — no rounding
16	0.48	0	Below midpoint — rounds DOWN
17	0.51	1	Above midpoint — rounds UP
34	1.02	1	Below midpoint — rounds DOWN
20000	600.0	600	Large exact value
100000000	3000000.0	3000000	Large safe integer

JSDoc Updated

The module-level JSDoc and the function-level JSDoc now accurately describe the Math.floor formula, include the half-up rationale, and list the new midpoint examples. The old "Math.round on cents arithmetic" wording has been removed.

3 · Migration 0009 — Ledger FK NOT NULL

Purpose

Every ledger_transaction must belong to a financial_transaction. The original Phase 4A schema left financial_transaction_id nullable with a comment noting it was "nullable so system/adjustment entries can exist". The hardening removes this loophole: the column is now NOT NULL at both the application layer and the PostgreSQL level.

Migration SQL (0009_ledger_tx_fk_not_null.sql)

```
-- Safety gate: abort if any orphan rows exist
DO $$
DECLARE orphan_count bigint;
BEGIN
  SELECT COUNT(*) INTO orphan_count
  FROM ledger_transactions
  WHERE financial_transaction_id IS NULL;
  IF orphan_count > 0 THEN
    RAISE EXCEPTION
      'Migration aborted: % rows have NULL financial_transaction_id.',
      orphan_count;
  END IF;
END $$;

-- No orphans confirmed — enforce NOT NULL
ALTER TABLE ledger_transactions
  ALTER COLUMN financial_transaction_id SET NOT NULL;
```

Migration Result

> ' **Applied successfully**
>
> Orphan check: 0 rows with NULL financial_transaction_id found (confirmed before ALTER)
>
> ALTER TABLE executed: financial_transaction_id is now NOT NULL in PostgreSQL

Registered in drizzle.__drizzle_migrations (id=10, created_at=1785862200000)

Re-running migrate is safe (ALTER is idempotent once column is already NOT NULL)

Schema Change

In lib/db/src/schema/index.ts, the ledgerTransactions table definition was updated:

```
// OLD
// financialTransactionId is nullable so system/adjustment entries can exist
// without a parent financial_transaction.
financialTransactionId: uuid("financial_transaction_id")
  .references(() => financialTransactions.id, { onDelete: "restrict" }),

// NEW
// Every ledger_transaction must belong to a financial_transaction.
// The FK is NOT NULL — enforced at application layer and DB layer.
financialTransactionId: uuid("financial_transaction_id")
  .notNull()
  .references(() => financialTransactions.id, { onDelete: "restrict" }),
```

Application Layer Change

(ledger.ts)

PostLedgerTransactionInput.financialTransactionId was changed from optional (?: string) to required (: string). A runtime guard was also added as a defence-in-depth check:

```
// Required field — DB column is NOT NULL
financialTransactionId: string;

// Guard in _postLedgerEntriesInTx()
if (!financialTransactionId) {
  throw new Error("financialTransactionId is required for every ledger_transaction");
}

// Insert (removed ?? null fallback)
financialTransactionId,  // was: financialTransactionId ?? null
```

4 · Ledger Ownership / Invariant Results

New Invariant Test

A new test was added to the "Ledger invariants" describe block:

```
it("no ledger_transaction has a NULL financial_transaction_id", async () => {
  const orphans = await db
    .select({ id: ledgerTransactions.id })
    .from(ledgerTransactions)
    .where(sql`${ledgerTransactions.financialTransactionId} IS NULL`);
  expect(orphans.length).toBe(0);
});
```

Live DB Verification

' Zero orphans in 134 ledger_transactions rows

SELECT COUNT(*) FROM ledger_transactions WHERE financial_transaction_id IS NULL !' 0

Total ledger_transactions rows: 134

All 134 rows have a valid financial_transaction_id reference

Business Rule Confirmations

- postCommitPrincipal is NOT exposed through any HTTP route — internal-only function
- Contributor fund commitment remains unimplemented as a public action in Phase 4A
- Only the authenticated contributor may commit their own funds (future rule preserved)
- Organizer-triggered commitment of another contributor's principal is not allowed
- fund_destination_type remains NULL in Phase 4A — unchanged
- organizer_bank_payout and dripjar_card destination values belong to later phases

5 · Test Results

> ' **301 tests passed, 1 pre-existing failure**
>
> Total: 302 tests across 13 test files
>
> Passed: 301
>
> Failed: 1 (pre-existing — phase3-automation deduplication, confirmed on base 65de175)
>
> New tests added in this hardening:
>
> Fee Engine: $0.50 midpoint, $1.50 midpoint, $5.00 exact, below/above midpoint,
>
> large safe-integer (100_000_000 !' 3_000_000)
>
> Ledger invariants: no ledger_transaction has NULL financial_transaction_id
>
> Invariant test fixes: 3 postLedgerTransaction calls given required financialTransactionId

Fee Rounding Test Suite (complete)

Test	Input (¢)	Expected
$200.00 !' $6.00 (exact)	20000	600
$100.00 !' $3.00 (exact)	10000	300
$1.00 !' $0.03 (exact)	100	3
$5.00 !' $0.15 (exact)	500	15
$0.50 midpoint !' rounds UP	50	2
$1.50 midpoint !' rounds UP	150	5
$0.16 below midpoint !' rounds DOWN	16	0
$0.34 below midpoint !' rounds DOWN	34	1
$0.17 above midpoint !' rounds UP	17	1

$0.33 above midpoint !' rounds UP

Large: $10,000.00	1000000	30000
Large safe-integer: $1,000,000.00	100000000	3000000
Zero principal !' zero fee	0	0
Throws on negative principal	-1	throws
Throws on non-integer principal	100.5	throws

Pre-Existing Failures (confirmed on 65de175 before any changes)

& phase3-automation deduplication test is flaky (pre-existing)

Test: "running twice does not double-count notifications (deduplication)"

Cause: Accumulated test jars in shared dev DB build up unaccepted agreements between test runs. Not caused by Phase 4A hardening code.

Confirmed: same failure on git stash (clean base commit 65de175)

& @types/pg missing from api-server tsconfig (pre-existing)

File: refresh-token-concurrency-proof.test.ts:30

Error: TS2307 Cannot find module 'pg'

Cause: @types/pg not in api-server devDependencies. Runtime unaffected.

Confirmed: same error on git stash (clean base commit 65de175)

6 · Typecheck & Build Results

lib/db Typecheck

> ' **pnpm tsc -p lib/db/tsconfig.json — 0 errors**
>
> lib/db rebuilt cleanly after schema change (financialTransactionId notNull)

API Server Typecheck

> **1 pre-existing error, 0 new errors**
>
> pnpm --filter @workspace/api-server run typecheck
>
> Error: refresh-token-concurrency-proof.test.ts:30 — TS2307 (cannot find "pg" types)

This error existed on 65de175 before Phase 4A hardening. No new errors introduced.

API Server Production Build

> ' **pnpm --filter @workspace/api-server run build — success**
>
> esbuild completed in 337ms
>
> dist/index.mjs + source map produced

No build errors

7 · Migration Verification

Full Migration Chain (0000 !’ 0009)

ID	Tag	Applied
1	0000_initial_schema	1785423140803
2	0001_sprint1_auth_hardening	1785423373313
3	0002_align_refresh_session_fk_name	1785424743904
4	0003_refresh_token_security	1785427879303
5	0004_add_refresh_family_default	1785429000000
6	0005_membership_lifecycle	1785778405941
7	0006_phase3_automation	1785780426979
8	0007_phase3_corrections	1785784000000
9	0008_phase4a_financial_ledger	1785811500000
10	0009_ledger_tx_fk_not_null	1785862200000

' Migration chain complete

pnpm --filter @workspace/db run migrate !’ "['] migrations applied successfully!"

All 10 migrations registered in drizzle.__drizzle_migrations

ledger_transactions.financial_transaction_id confirmed NOT NULL in information_schema

psql \d ledger_transactions shows: financial_transaction_id | uuid | not null

8 · Architecture Review

Fee Rounding

Integer overflow
SAFE — principalCents * rateBps + 5_000 stays well within Number.MAX_SAFE_INTEGER for all realistic drip amounts (100_000_000 * 300 + 5000 = 30_000_005_000 << 9×10^15)

Floating-point correctness
IMPROVED — new formula uses integer arithmetic only; no FP division until the final floor which is exact for integer-valued doubles

Half-up semantics
CORRECT — adding 5_000 (half of 10_000 divisor) guarantees the midpoint (1.5¢ boundary) rounds UP

Existing test compatibility
All 15 fee tests pass, including previously passing Math.round cases (behavior identical for all non-midpoint values)

Ledger Ownership

Orphan prevention
ENFORCED — financialTransactionId is required at three layers: TypeScript type (: string), runtime guard (throws before insert), PostgreSQL NOT NULL constraint

Migration safety
SAFE — orphan check aborts migration if any NULL rows found; in practice zero orphans existed (confirmed by pre-migration query)

All existing postings
VERIFIED — 134 existing ledger_transactions rows all have non-NULL financial_transaction_id; zero orphans

Double-entry integrity
INTACT — all balance invariants pass; no ledger transaction created without a parent financial_transaction

Idempotency
PRESERVED — idempotency key logic unchanged; ALTER TABLE is safe to re-run

Business Rules Unchanged

- postCommitPrincipal is not reachable from any HTTP route
- Contributor fund commitment is not a public action in Phase 4A
- fund_destination_type is NULL in Phase 4A — no changes made
- organizer_bank_payout and dripjar_card belong to Phase 4B+
- No Stripe SDK, PaymentIntents, or any provider API was touched

9 · Commit, Push & Verification

Final Commit

```
Hash:  d63d0f9
Title: Phase 4A hardening: fee rounding + ledger FK NOT NULL

Body:
  - Replace Math.round with explicit half-up integer formula
    Math.floor((principalCents * rateBps + 5_000) / 10_000)
  - Add midpoint, below/above midpoint, and large safe-integer fee tests
  - Make ledger_transactions.financial_transaction_id NOT NULL
  - Add migration 0009 with orphan-safety gate
  - Require financialTransactionId in PostLedgerTransactionInput
  - Add ledger FK NOT NULL invariant test

Parent: b6fd453 — Update agent memory with Phase 4A learnings
Files:  6 changed, 130 insertions(+), 25 deletions(-)
```

Push Result

' Pushed successfully to origin/main

Remote: https://github.com/JB102298/TripJar.git

Branch: main

Result: Pushed to main on github

Remote Verification

' origin/main confirmed at d63d0f9

git log --oneline -4:

d63d0f9 (HEAD -> main, origin/main, origin/HEAD) Phase 4A hardening: fee rounding + ledger FK NOT NULL

b6fd453 Update agent memory with Phase 4A learnings

8841ec9 Add DripJar financial ledger foundation

65de175 Add M3Jar cutoff and reminder automation

Working-Tree Status

Clean for committed work

Staged/modified tracked files: none (all Phase 4A hardening files committed)

Untracked files: attached_assets/ (spec input files, not committed by design)

phase4a-report.pdf (previous session report, not committed)

.replit and .agents/ auto-managed by platform — not part of code history

10 · Blockers & Warnings

No blockers encountered during this hardening session. Two pre-existing warnings carried forward from the Phase 4A foundation commit (confirmed unchanged on base 65de175):

& Warning 1 — @types/pg missing (pre-existing)

refresh-token-concurrency-proof.test.ts:30 — TS2307 cannot find "pg" types

Fix: add @types/pg to artifacts/api-server/package.json devDependencies

Impact: typecheck exits code 2; runtime behavior is unaffected

& Warning 2 — Automation deduplication test is flaky (pre-existing)

phase3-automation.test.ts "running twice does not double-count" fails intermittently

Cause: accumulated test jars in shared dev DB accumulate unaccepted agreements

Fix: cancel test jars in afterAll OR filter automation to Saving-status jars only

11 · Final Confirmations

' No Stripe / provider work occurred

Zero Stripe SDK, PaymentIntents, SetupIntents, webhook, or provider API calls added or modified.

' No production / external infrastructure changed

No changes to production DB, DNS, Resend, Cloudflare, Expo/EAS, secrets, or deployment configuration.

' History not rewritten

b6fd453 was not amended or rebased. d63d0f9 is a clean new commit on top of it.

' All Phase 4A invariants remain intact

Every posted ledger transaction is balanced (debits = credits)

No ledger entry has a zero or negative amount

No ledger_transaction has a NULL financial_transaction_id (0 of 134)

Simulated contributions never appear in ledger balances

Refund/commit cannot exceed refundable principal